

**CSE/ISyE 6740**  
**Computational Data Analysis**

# **Neural Networks**

09/30/2026

Kai Wang, Assistant Professor in Computational Science and Engineering

[kwang692@gatech.edu](mailto:kwang692@gatech.edu)

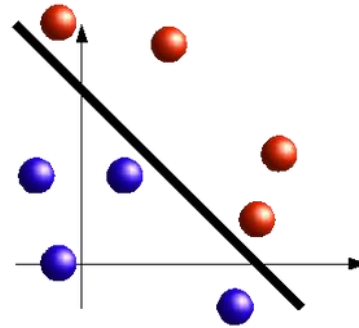
# Outline

- **Supervised Learning**
  - **Neural networks**
    - Perceptron
    - Multilayer perceptron
    - Backpropagation

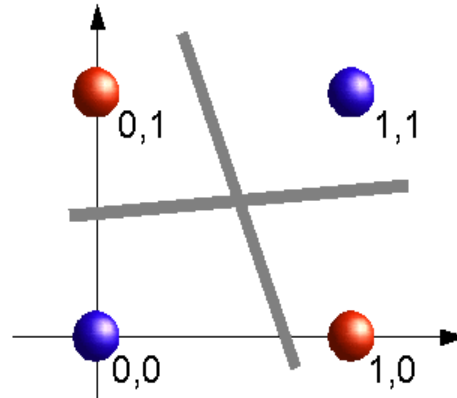
# Neural Networks

# Learning Non-linear Decision Boundary

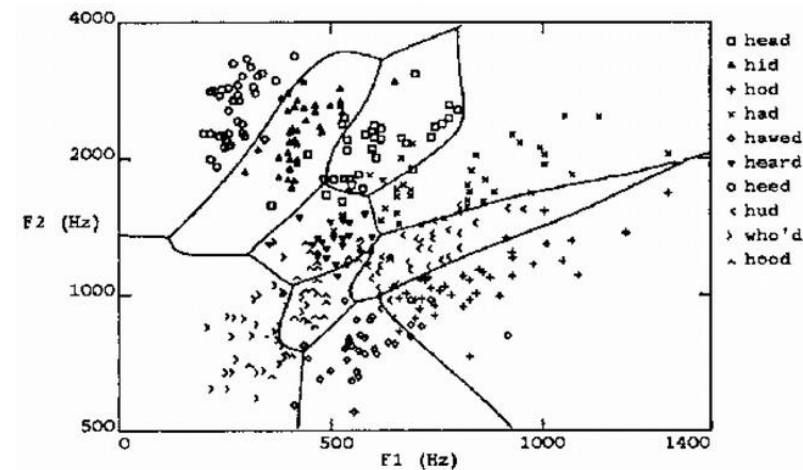
- Linear separable



- Non-linear separable



XOR gate



Speech recognition

# Learning Non-linear Decision Boundary

- Linear separable

- $f(x) = w^\top x$

- Linear regression: MSE loss:  $l(x) = l^{MSE}(f(x), y) := \|y - f(x)\|^2$

- Logistic regression: logistic loss  $l(x) = l^{logistic}(f(x), y) := \log(1 + e^{-yf(x)})$

- Support vector machine: hinge loss  $l(x) = l^{HINGE}(f(x), y) := \begin{cases} 0. & \text{if } yf(x) \geq 1 \\ 1 - yf(x). & \text{otherwise} \end{cases}$

- Non-linear separable

- $f(x) = w^\top \phi(x)$

- Kernel regression

- Gaussian process regression

- $\phi(x)$  is too big so we don't compute  $\phi(x)$  and  $\theta$  at all.

- We only compute  $k(x, x') = \phi(x)^\top \phi(x')$

- **Neural networks:** let's just learn all  $w$  and  $\phi(x)$  in the parametric model!

# Function Approximator

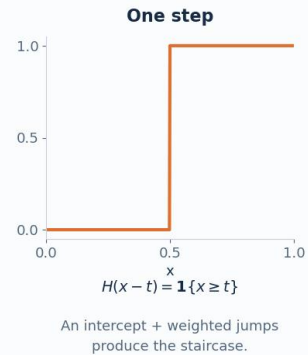
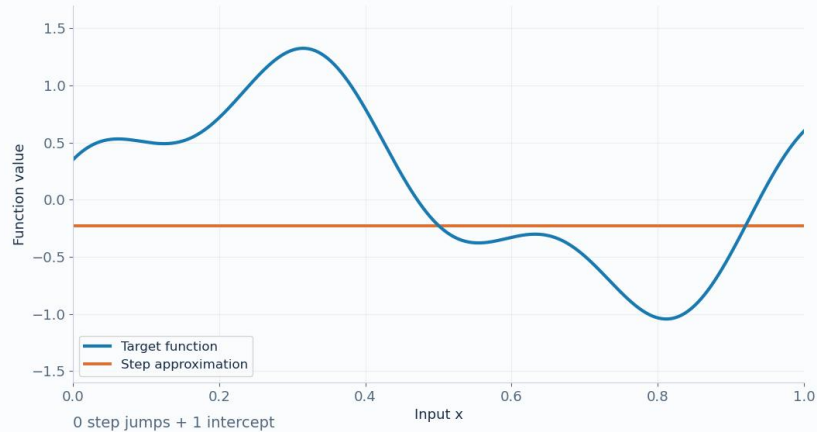
Using a piecewise-constant function to approximate non-linear functions

## Step functions approximate a 1D function

RMSE 0.7806

1 interval per input axis

1 adjustable coefficients



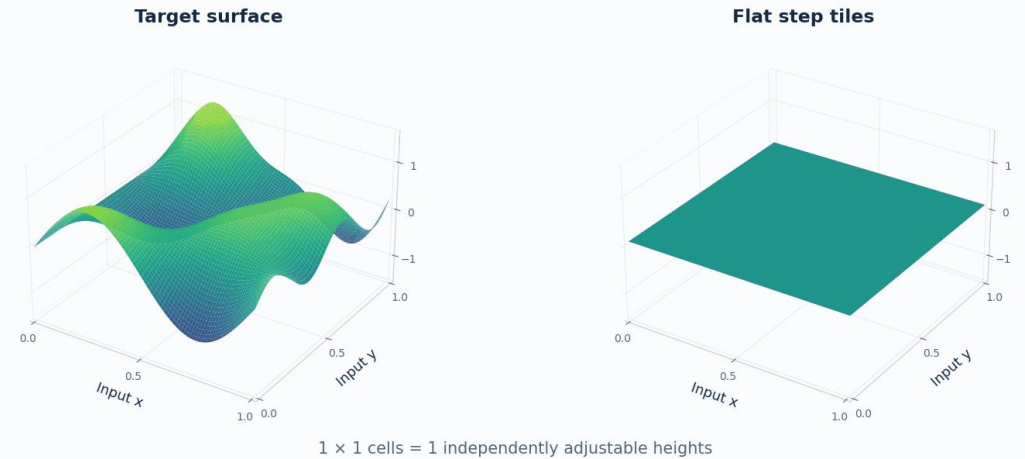
Fixed grid and basis shapes. Counts refer to adjustable output coefficients.

## Step functions approximate a 2D function

RMSE 0.4509

1 interval per input axis

1 adjustable coefficients



Fixed grid and basis shapes. Counts refer to adjustable output coefficients.

# Function Approximator

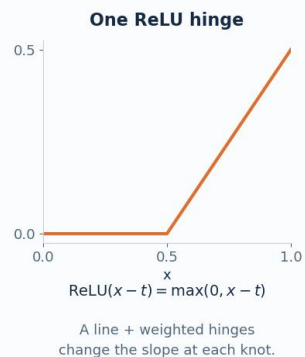
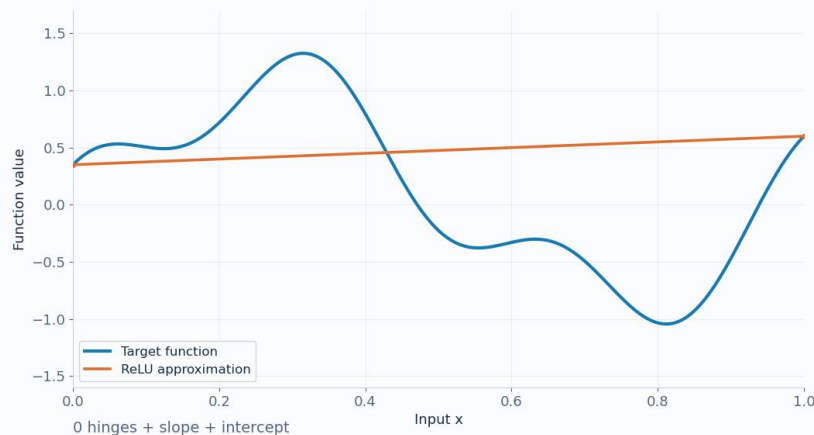
Using a piecewise-linear function to approximate non-linear functions

## ReLU functions approximate a 1D function

1 interval per input axis

2 adjustable coefficients

RMSE 0.8269



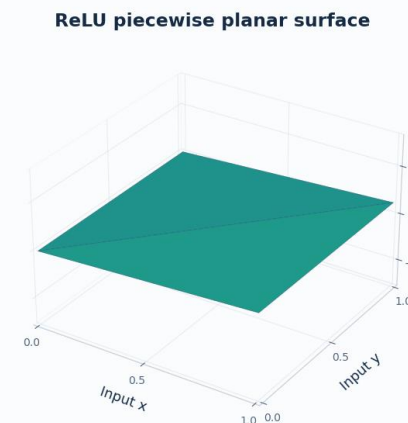
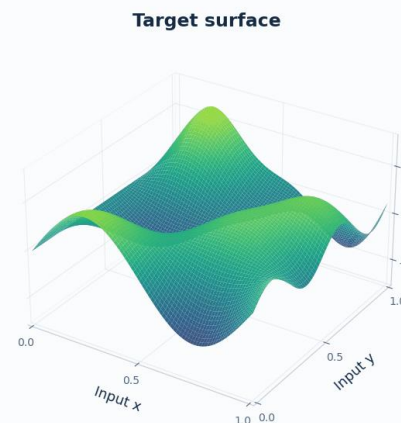
Fixed grid and basis shapes. Counts refer to adjustable output coefficients.

## ReLU functions approximate a 2D function

1 interval per input axis

4 adjustable coefficients

RMSE 0.4537



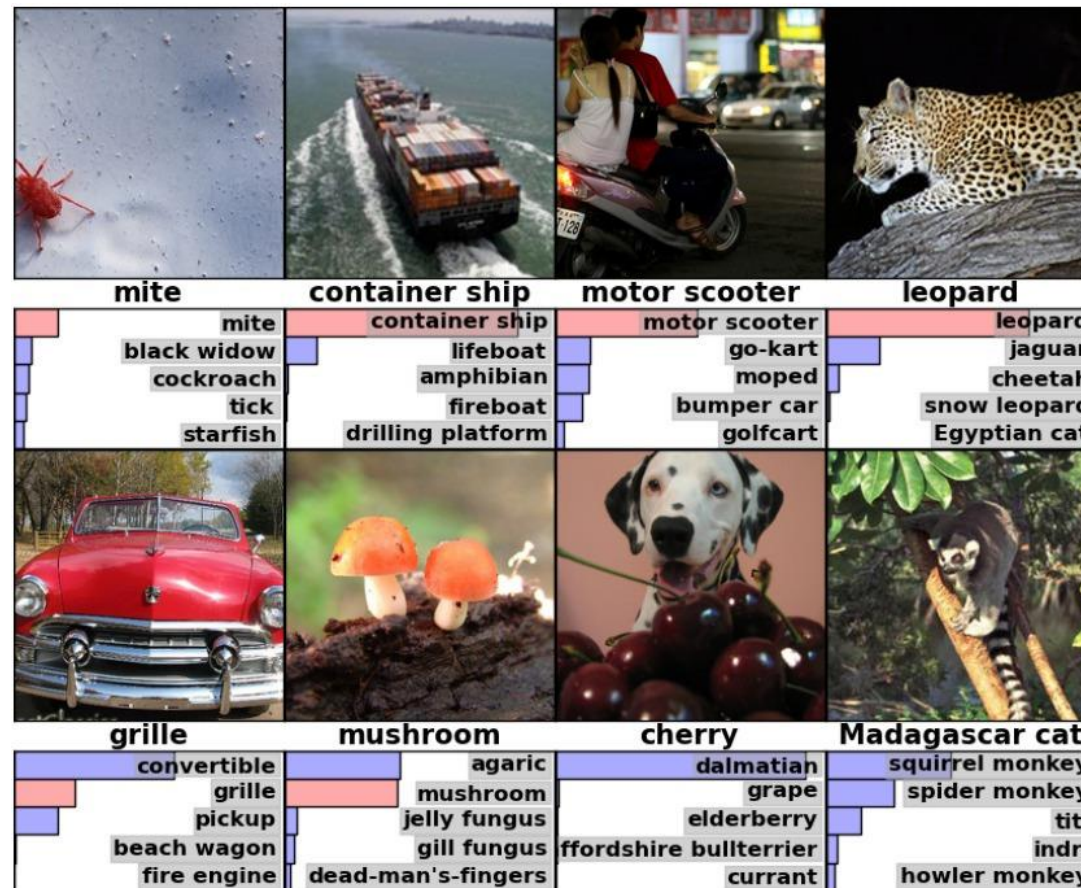
$(1 + 1)^2 = 4$  vertex coefficients / 2 planar triangles

Fixed grid and basis shapes. Counts refer to adjustable output coefficients.

What if the dimensionality  $x \in \mathbb{R}^d$  grows? The number of parameters explodes very quickly

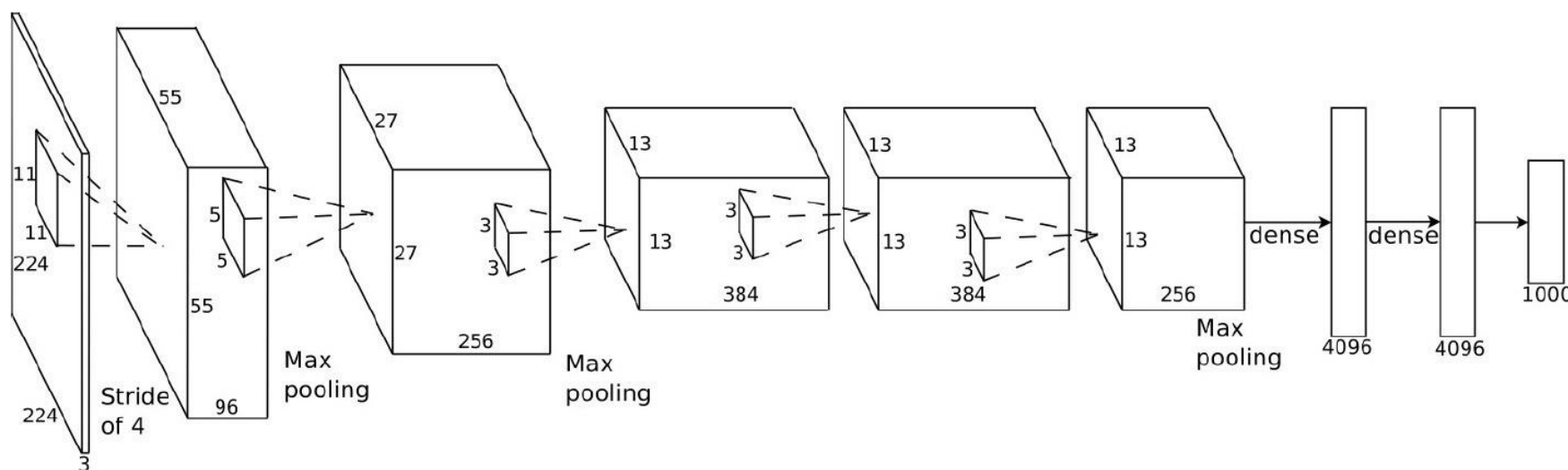
# ImageNet

- Image classification with 1.3M color images and 1000 classes need large scale nonparametric methods



# Neural Networks and Convolution Features

- 8 layer convolution neural network [[Krizhevsky et al. 2012](#)] achieved the state-of-the-art result (beating the second place by 10%).
  - **First 5 layers:** **convolution** + max pooling
  - **Next 2 layers:** **fully connected nonlinear neurons**
  - **Last layer:** multiclass logistic regression



# Biologically Inspired Learning Model

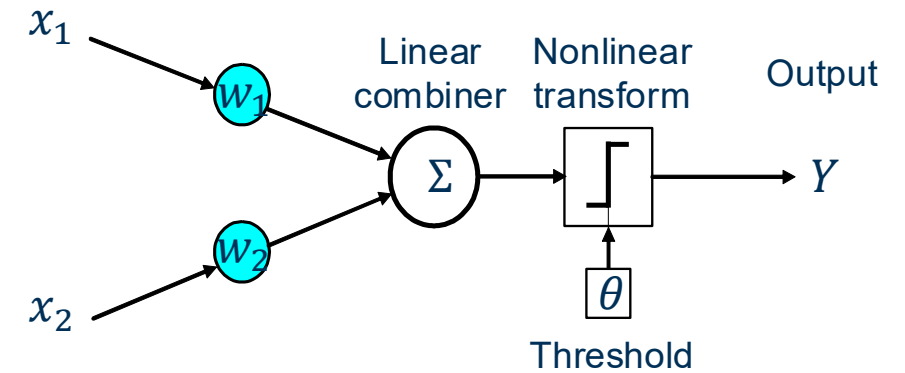
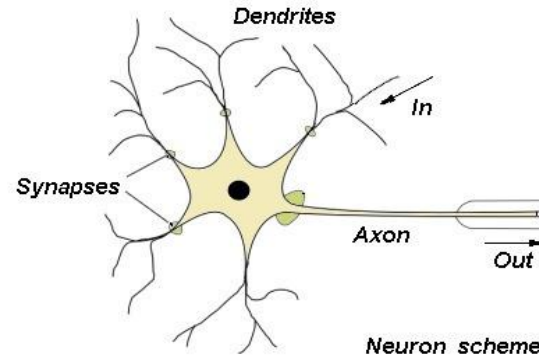
- Consider humans:
  - Neuron switching time  $\sim 0.001$  second
  - Number of neurons  $\sim 10^{10}$
  - Connections per neuron  $\sim 10^{4-5}$
  - Scene recognition time  $\sim 0.1$  second
  - Much parallel computation



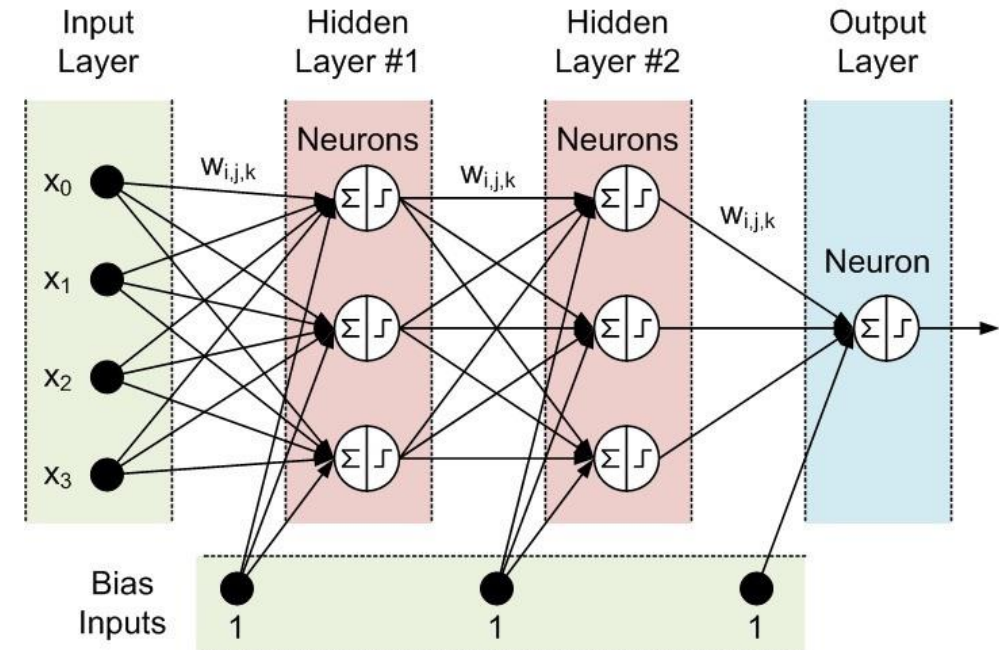
# Perceptron

# Perceptron

- From biological neuron to artificial neuron (perceptron)



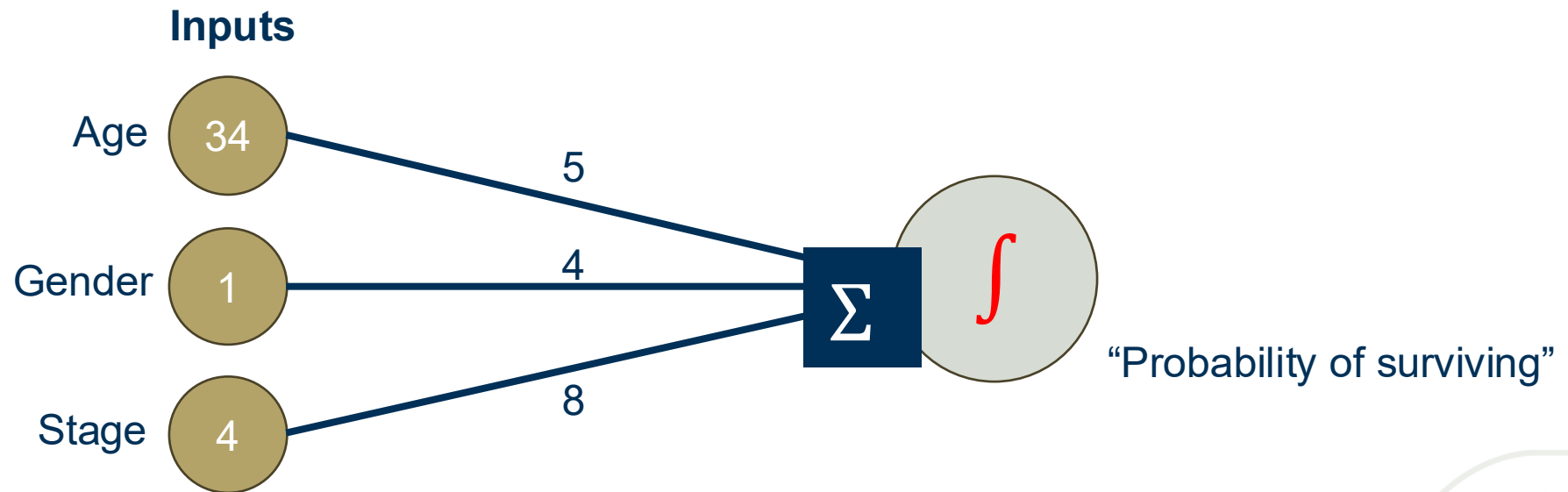
- Artificial neural nets (ANN)
  - Many neuron-like threshold switching units
  - Many weighted interconnections among units
  - Highly parallel, distributed processes



# Logistic Regression Model is a Perceptron

- **Independent variable** = input variable
- **Dependent variable** = output variable
- **Coefficients** = “weights”

Logistic regression model (the sigmoid unit) is a perceptron



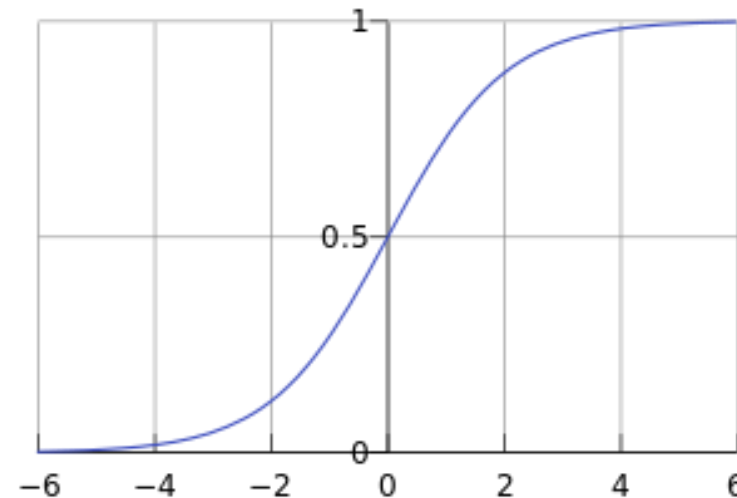
# Logistic Regression

- Assume that the posterior distribution  $p(y = 1|x)$  takes a particular form

$$p(y = 1|x, w) = \frac{1}{1 + \exp(-w^T x)}$$

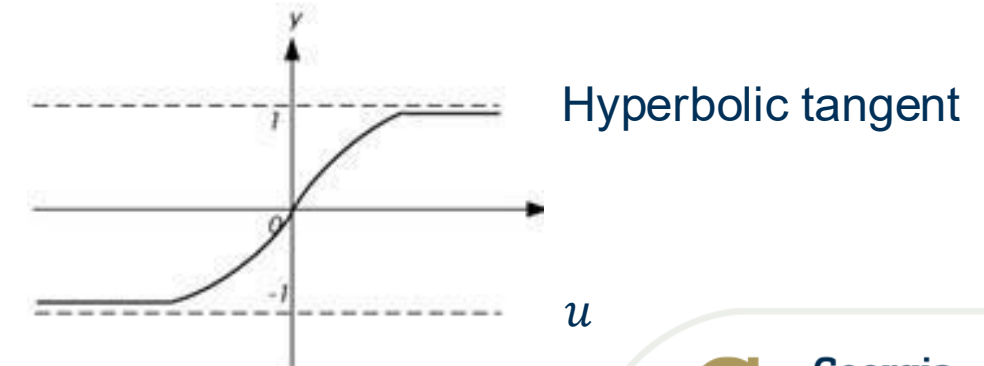
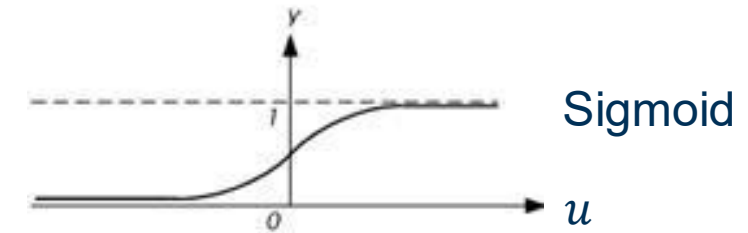
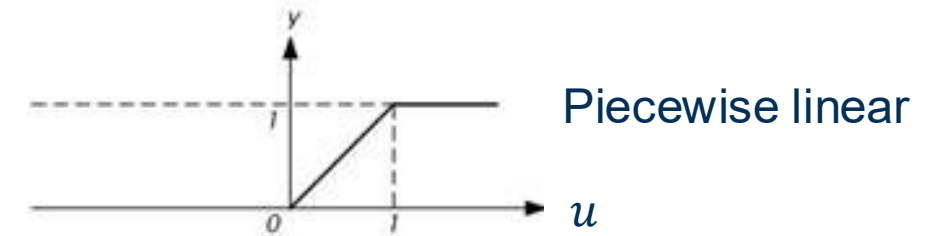
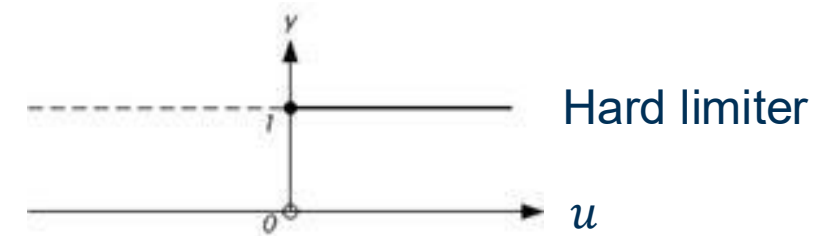
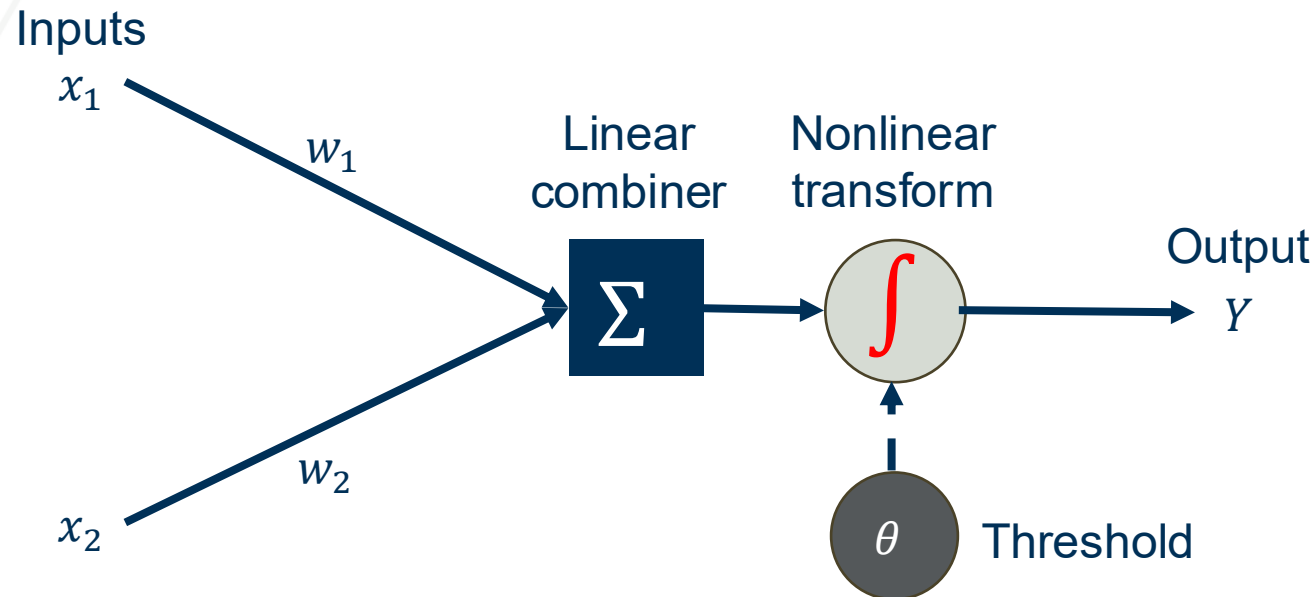
- Logistic function (or sigmoid function)  $\sigma(x) = \frac{1}{1 + \exp(-u)}$

- $\frac{\partial \sigma(u)}{\partial u} = \sigma(u)(1 - \sigma(u))$



# Other Nonlinear Neurons

- Use different nonlinear transformations  $f(u)$
- Before that, perform weighted combination of inputs  $u = w^T x$



# Learning with Square Loss

- Find  $w$ , such that the conditional probability of the labels are close to the actual labels (may be values other than 0 or 1)

$$\min_w l(w) := \sum_{i=1}^n \left( y^i - P(y = 1 | x^i, w) \right)^2 = \sum_{i=1}^n \left( y^i - \sigma(w^\top x^i) \right)^2$$

- **NOT** a convex objective function
- Use gradient descent to find a local optimum

# The Gradient of $l(w)$

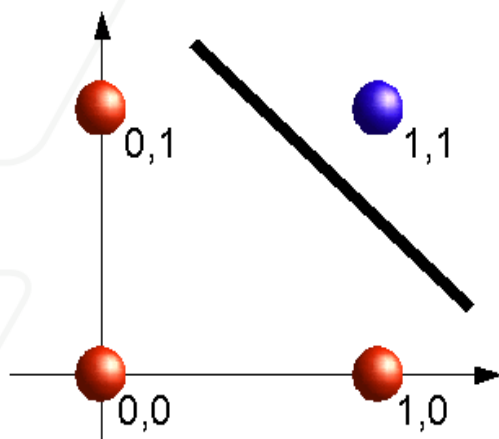
$$l(w) := \sum_{i=1}^n (y^i - \sigma(w^\top x^i))^2$$

- Let  $u^i = w^\top x^i$
- For sigmoid function:  $\frac{\partial \sigma(u)}{\partial u} = \sigma(u)(1 - \sigma(u))$

- Gradient

$$\frac{\partial l(w)}{\partial w} = \sum_{i=1}^n 2 (y^i - \sigma(u^i)) \sigma(u^i) (1 - \sigma(u^i)) x^i$$

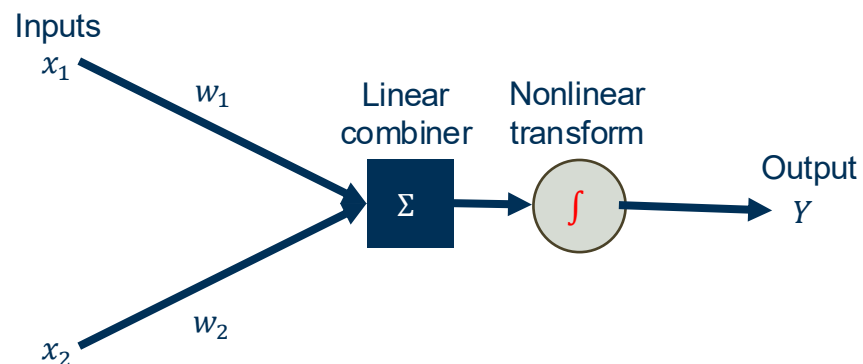
# Decision Boundary of a Perceptron



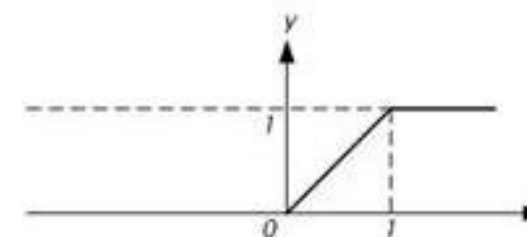
| x | y | Z (color) |
|---|---|-----------|
| 0 | 0 | 0         |
| 0 | 1 | 0         |
| 1 | 0 | 0         |
| 1 | 1 | 1         |

| $u$  | $P(y x)$ | $Z'$ |
|------|----------|------|
| 0    | 0        | 0    |
| 0.35 | 0.35     | 0    |
| 0.2  | 0.2      | 0    |
| 0.55 | 0.55     | 1    |

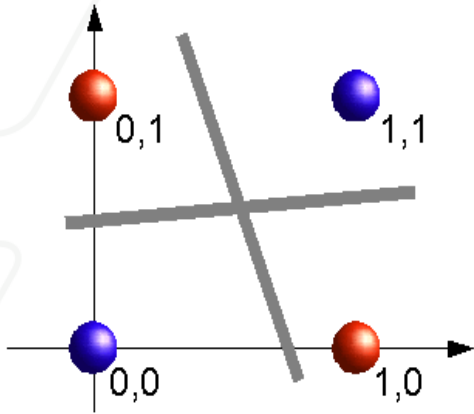
$$\theta = 0.5$$



**Weights:**  $w_1 = 0.2, w_2 = 0.35$   
**Nonlinear transform:** piecewise linear  
**Threshold:**  $\theta = 0.5$

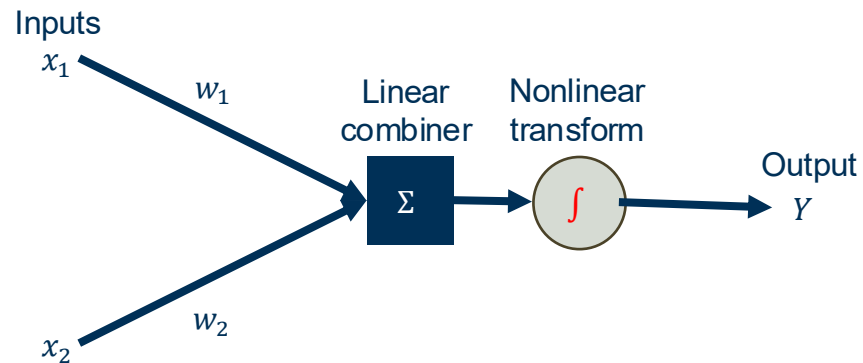


# Difficult Problem for Perceptron



| x | y | Z (color) |
|---|---|-----------|
| 0 | 0 | 1         |
| 0 | 1 | 0         |
| 1 | 0 | 0         |
| 1 | 1 | 1         |

| $\Sigma$ | $P(y x)$ | $Z'$ |
|----------|----------|------|
|          |          |      |
|          |          |      |
|          |          |      |
|          |          |      |

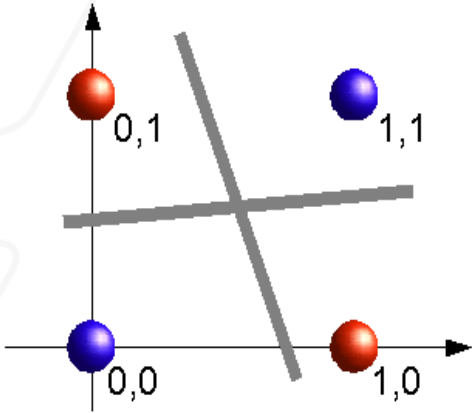


**Weights:**  $w_1 = ?$ ,  $w_2 = ?$

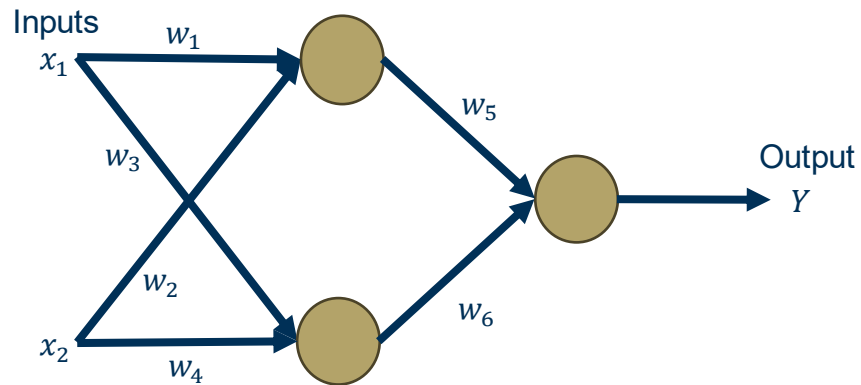
**Nonlinear transform:** any monotone function

**Threshold:** any threshold

# Connecting Perceptrons = Neural Networks!!



| x | y | Z (color) |
|---|---|-----------|
| 0 | 0 | 1         |
| 0 | 1 | 0         |
| 1 | 0 | 0         |
| 1 | 1 | 1         |



A possible set of weights:

$$(w_1, w_2, w_3, w_4, w_5, w_6) = (0.6, -0.6, -0.7, 0.8, 1, 1)$$

# Expressive Capabilities of Artificial Neural Networks (ANNs)

- **Boolean functions:**

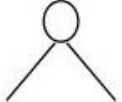
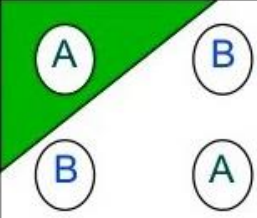
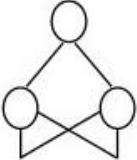
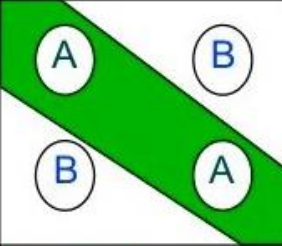
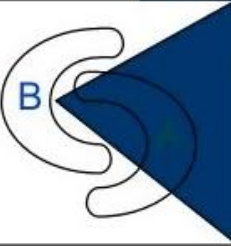
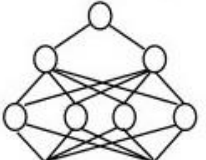
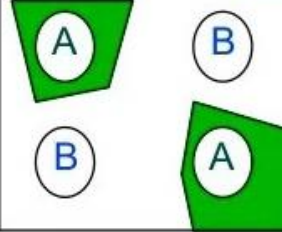
- Every Boolean function can be represented by network with a single hidden layer
- But might require exponential (in number of inputs) hidden units

- **Continuous functions:**

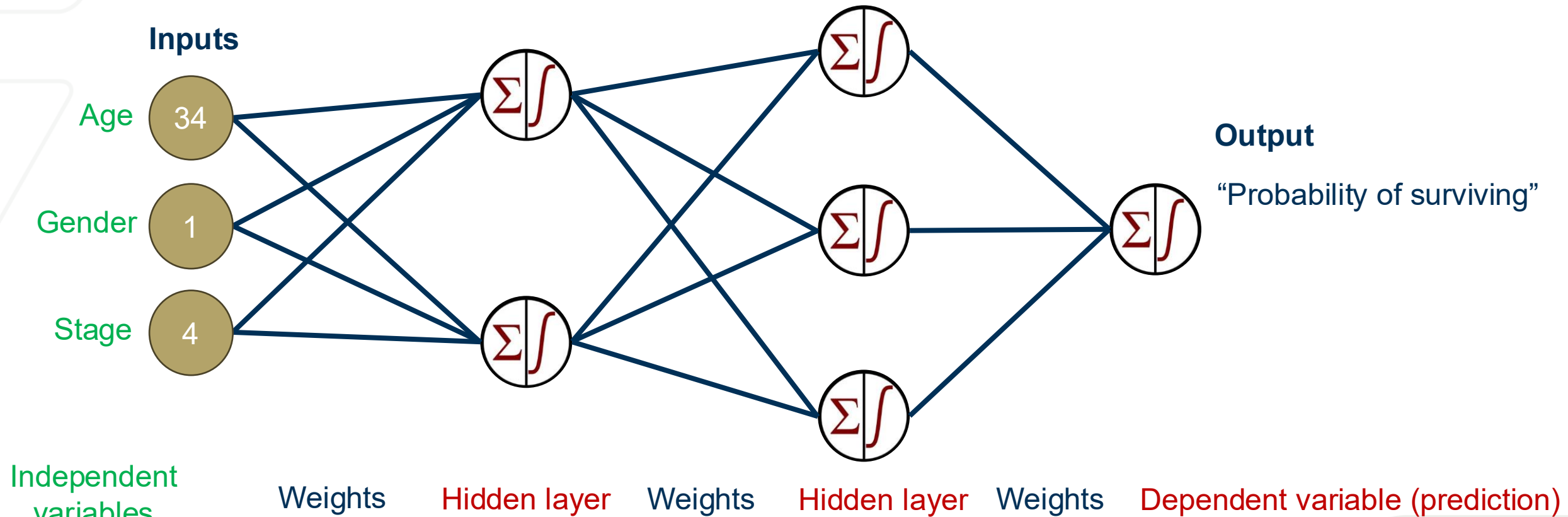
- Every bounded continuous function can be approximated with arbitrary small error, by network with one hidden layer [Cybenko 1989; Hornik et al. 1989]
- Any function can be approximated to arbitrary accuracy by a network with two hidden layers [Cybenko 1988]

# Multilayer Perceptron

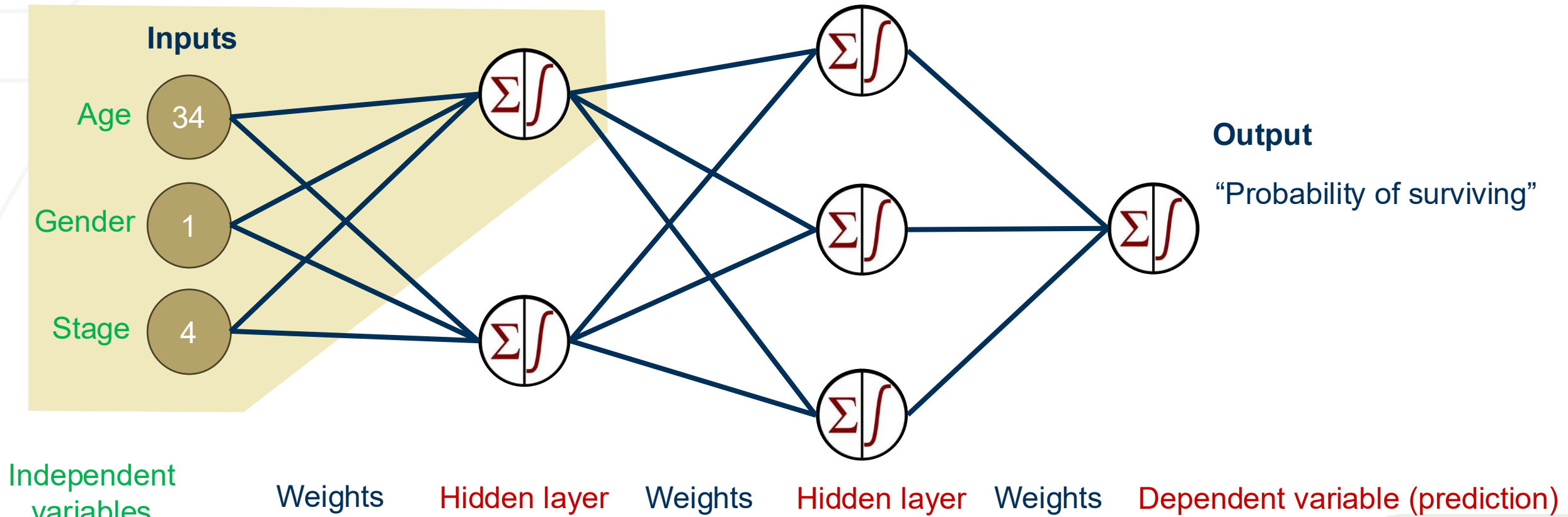
# Multilayer Perceptron

| Structure                                                                                          | Types of Decision Regions                      | Exclusive-OR Problem                                                                  | Classes with Meshed regions                                                           | Most General Region Shapes                                                            |
|----------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Single-Layer<br>  | Half Plane Bounded By Hyperplane               |    |    |    |
| Two-Layer<br>     | Convex Open Or Closed Regions                  |    |    |    |
| Three-Layer<br> | Arbitrary (Complexity Limited by No. of Nodes) |  |  |  |

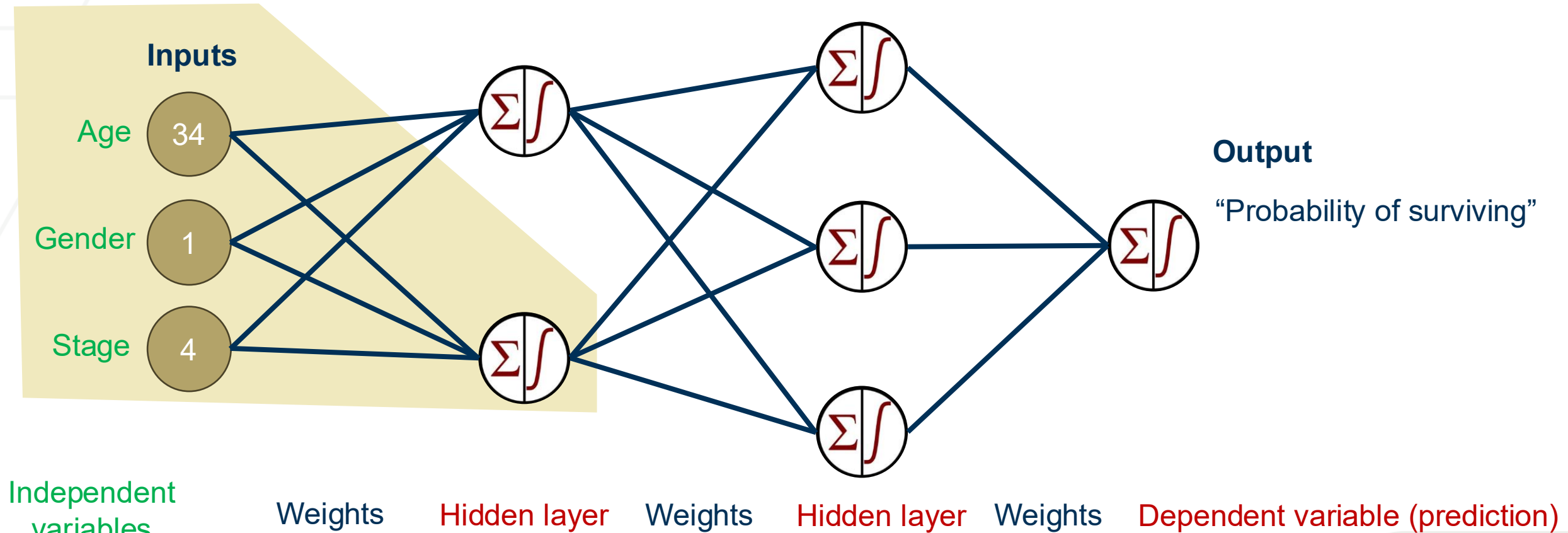
# Neural Network Model



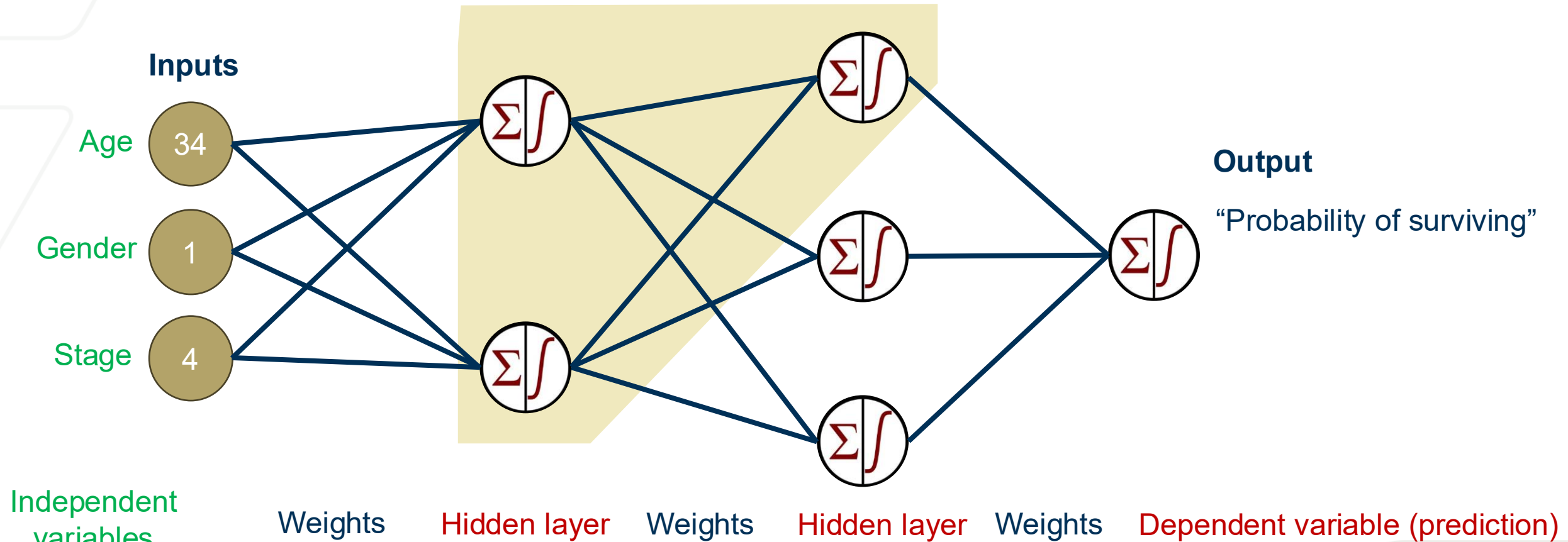
# “Combined Logistic Models”



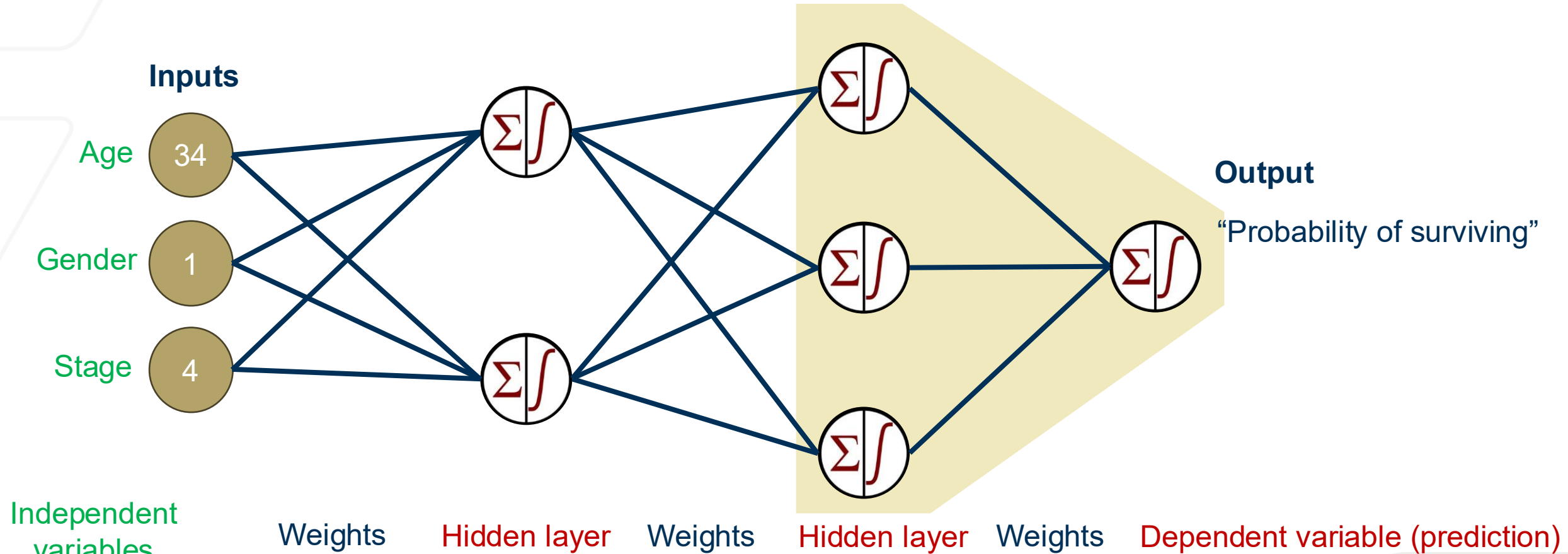
# “Combined Logistic Models”



# “Combined Logistic Models”

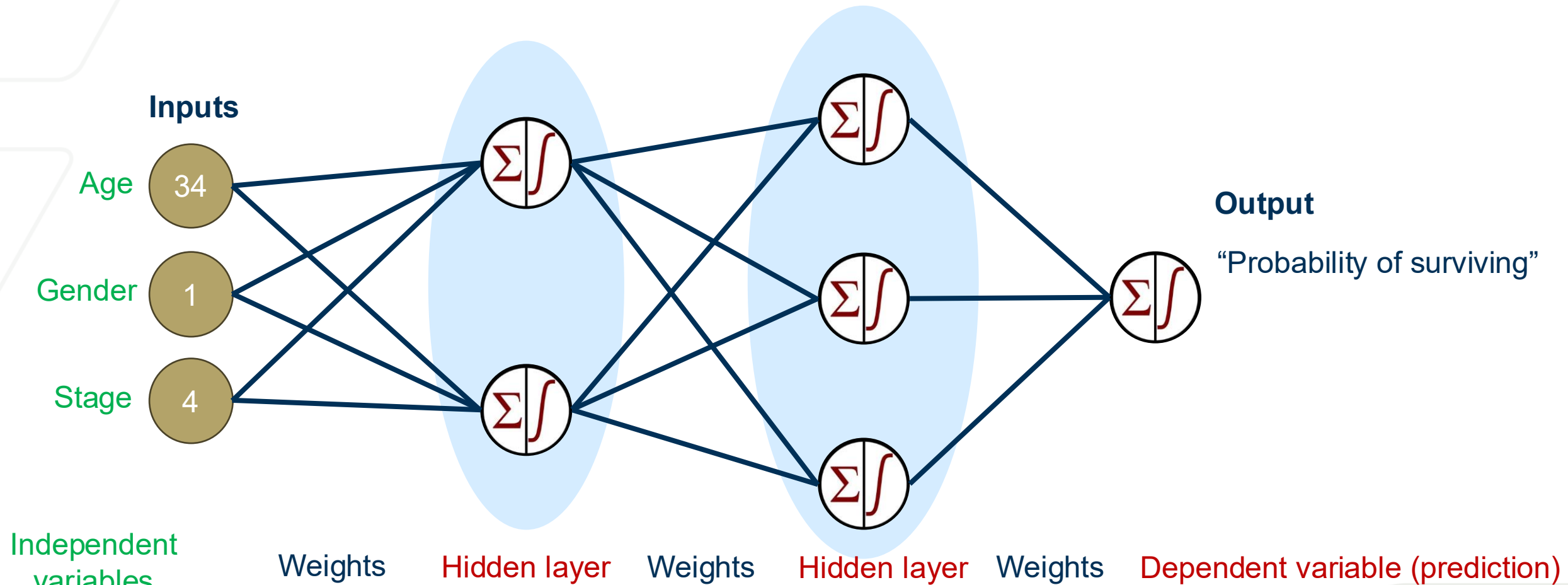


# “Combined Logistic Models”

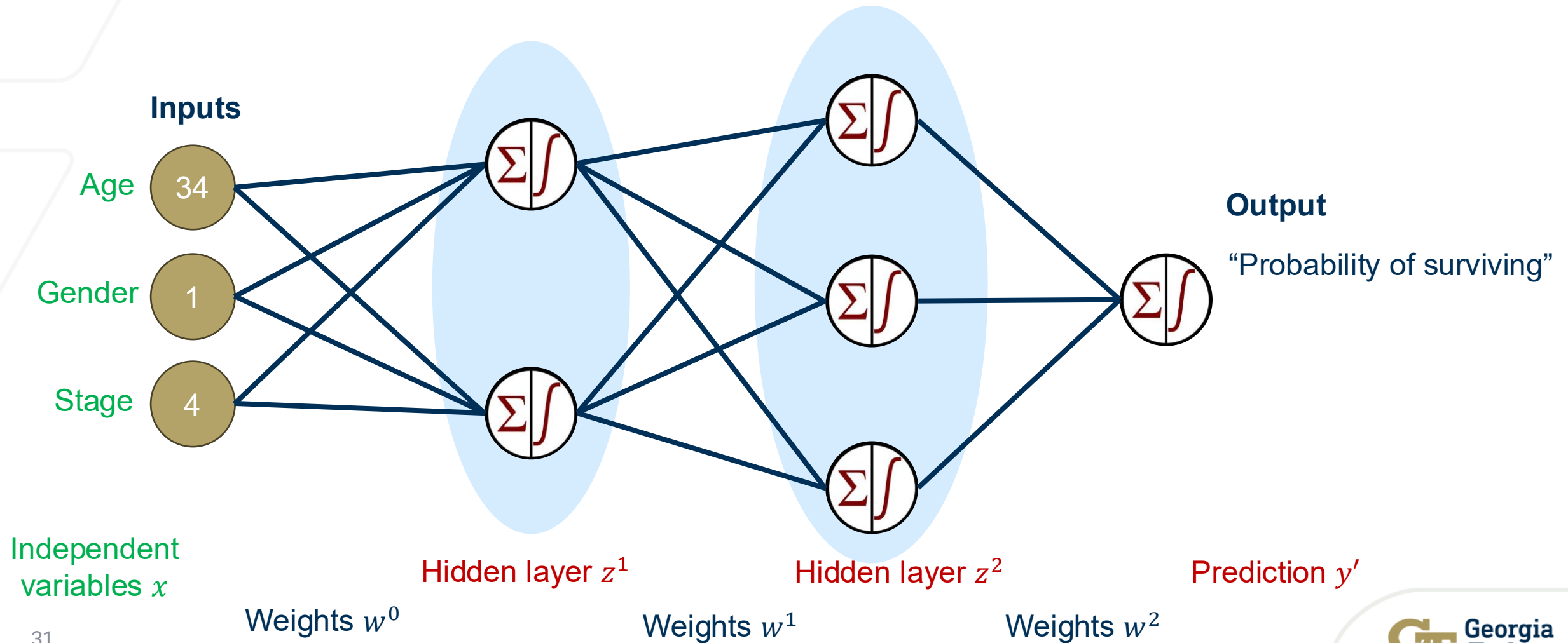


# Backpropagation

# How to Train the Hidden Units?



# How to Train the Hidden Units?



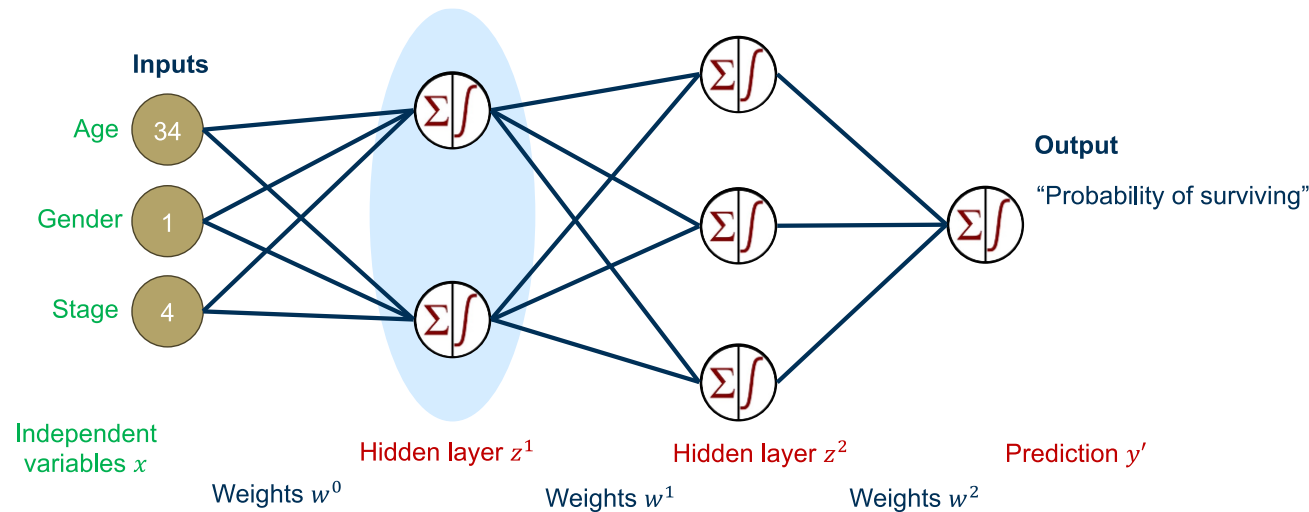
# How to Train the Hidden Units?

- Computing the loss function by **forward pass** (suppose using sigmoid function  $\sigma$ )

$$l(w^0, w^1, w^2) = \sum_{i=1}^n (y^i - y')^2$$

Where

- $y' = \sigma(w^{2\top} z^2)$
- $z_i^2 = \sigma(w_i^{1\top} z^1) \quad \forall i$
- $z_i^1 = \sigma(w_i^{0\top} x) \quad \forall i$



- NOT** a convex objective function
- Use gradient descent to find local optimum

# Compute the Gradient

- First, what is the gradient of  $w^2$ ?
  - Consider this as a logistic regression problem in the last layer.

- Input:**  $z^2 \in \mathbb{R}^{n_2}$

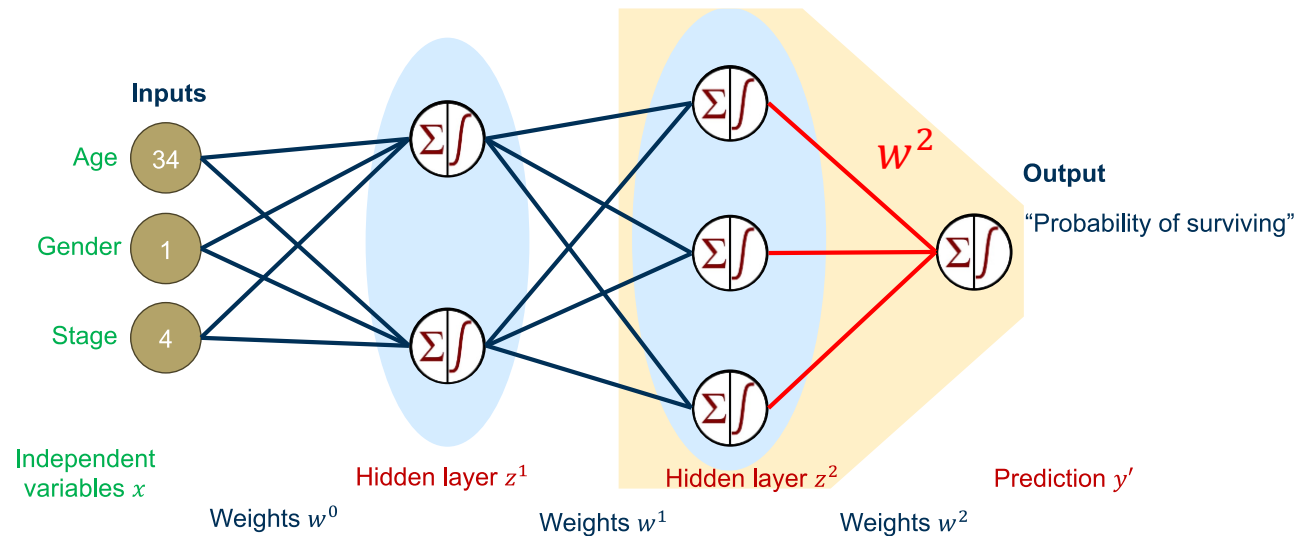
- Output:**  $y' = \sigma(w^{2\top} z^2)$

- Loss:**

$$l(w^0, w^1, w^2) = \sum_{i=1}^n (y^i - y')^2 = \sum_{i=1}^n \left( y^i - \sigma(w^{2\top} z^2) \right)^2$$

Let  $u^2 = w^{2\top} z^2$

$$\frac{\partial l}{\partial w^2} = \sum_i 2 \left( y^i - \sigma(u^2) \right) \sigma(u^2) (1 - \sigma(u^2)) z^2$$



# Compute the Gradient

- Can we compute the gradient of  $w^1$  and  $w^0$ ?
- We can use chain rule!
- Gradient:

$$\frac{\partial l}{\partial w^1} = \underbrace{\frac{\partial l}{\partial z^2}}_{\mathbb{R}^{1 \times \dim z^2}} \underbrace{\frac{\partial z^2}{\partial w^1}}_{\mathbb{R}^{\dim z^2 \times \dim w^1}} \in \mathbb{R}^{1 \times \dim w^1}$$

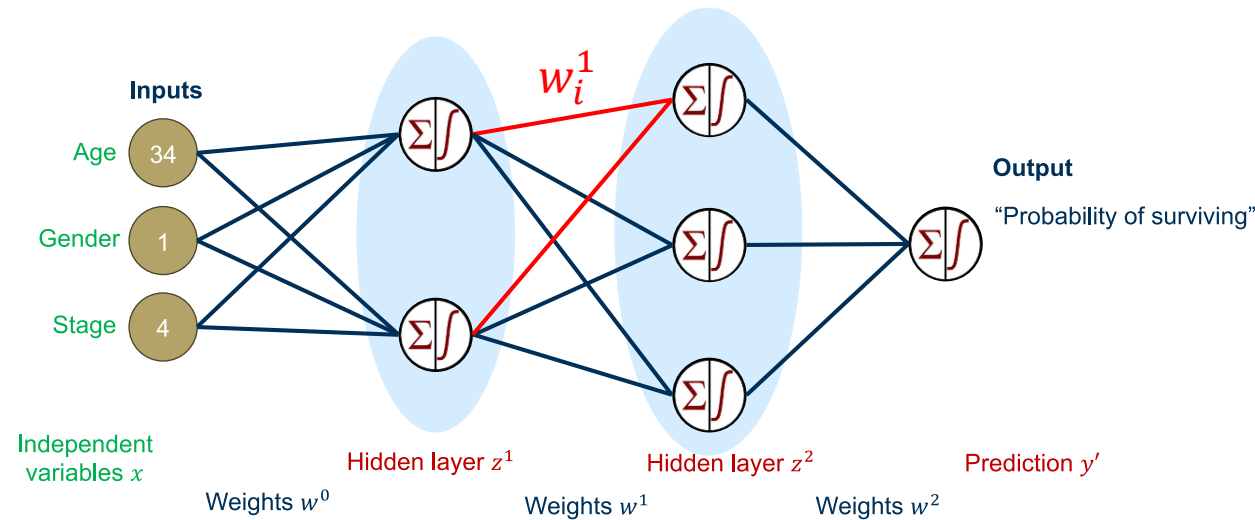
$$\frac{\partial l}{\partial z^2} = \sum_i 2 \left( y^i - \sigma(u^2) \right) \sigma(u^2) (1 - \sigma(u^2)) w^2$$

$$\frac{\partial z_i^2}{\partial w_i^1} = \frac{\partial \sigma(w_i^{1\top} z^1)}{\partial w_i^1} = \frac{\partial \sigma(v_i^2)}{\partial w_i^1} = \sigma(v_i^2) (1 - \sigma(v_i^2)) z^1 \quad \text{where } v_i^2 = w_i^{1\top} z^1 \in \mathbb{R}$$

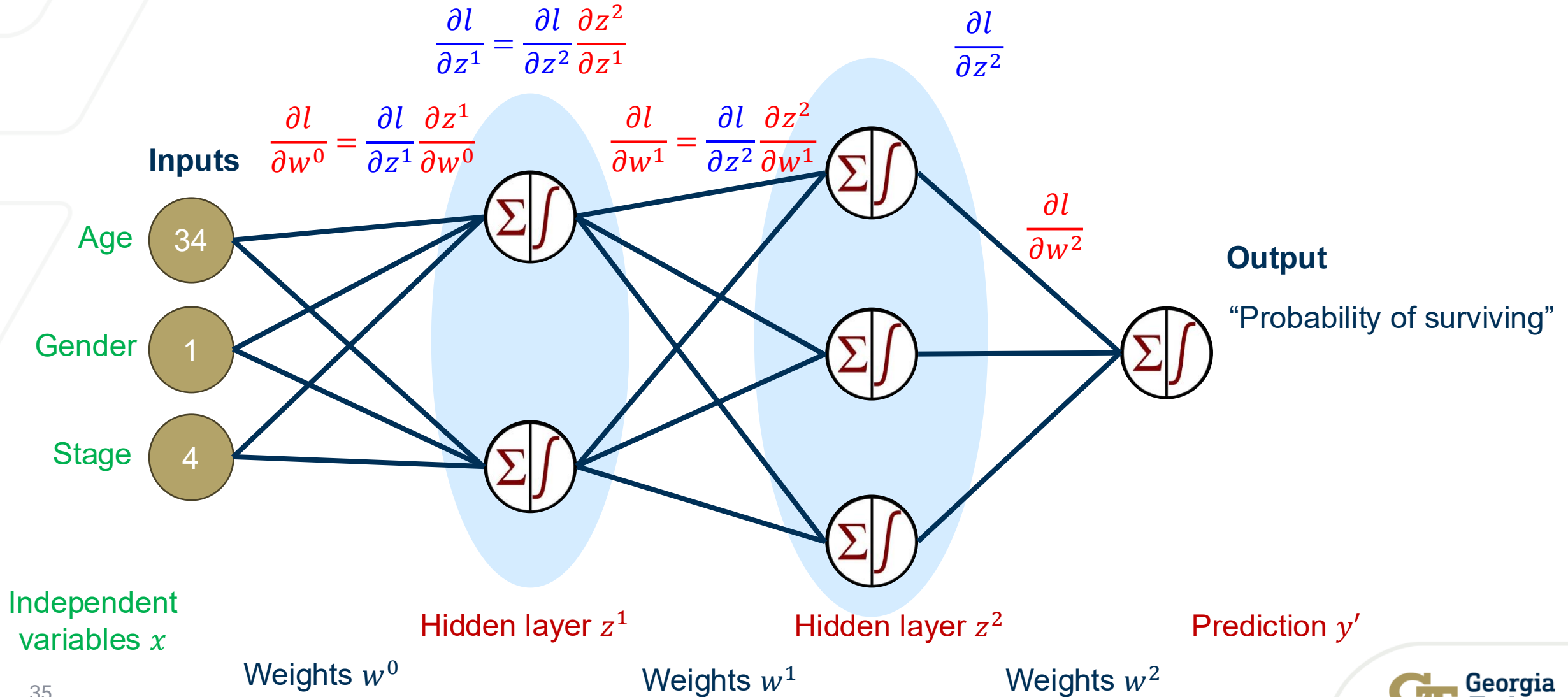
$$y' = \sigma(w^{2\top} z^2)$$

$$z_i^2 = \sigma(w_i^{1\top} z^1) \quad \forall i$$

$$z_i^1 = \sigma(w_i^{0\top} x) \quad \forall i$$

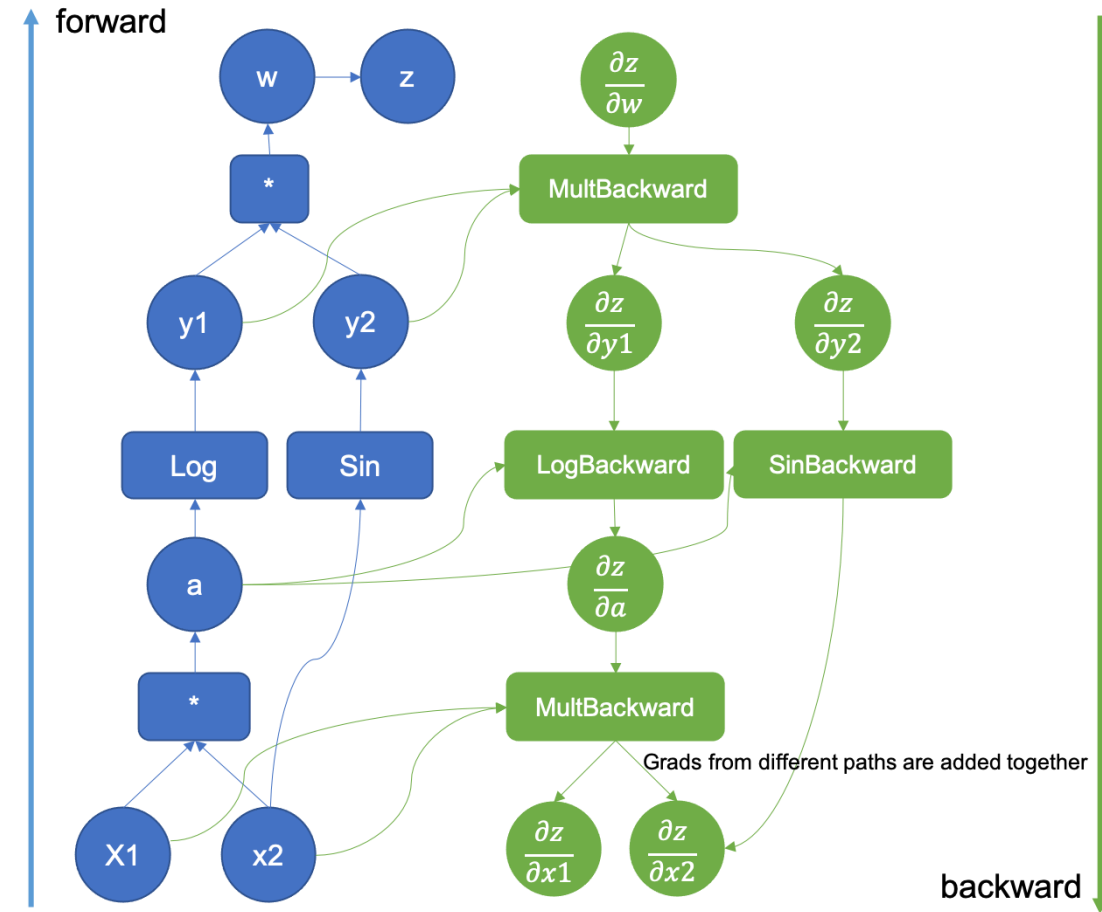


# Compute the Gradient (Backpropagation)



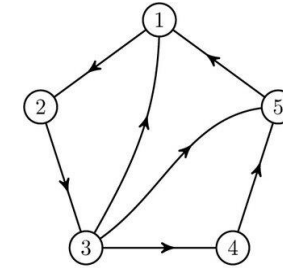
# Computation Graph and Auto-differentiation

- **Forward pass:**
  - Define the loss function and build the computation graph.
- **Backward pass:**
  - During backpropagation, we start from the loss function to backtrace through all the nodes.
    - For each node  $z$  in the graph, compute  $\frac{\partial l}{\partial z}$  by chain rule.
    - We only need to keep track of  $\frac{\partial l}{\partial z}$  for the node.
    - Compute  $\frac{\partial l}{\partial w}$  by chain rule using  $\frac{\partial l}{\partial z}$

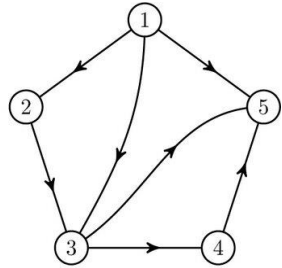


# Questions

- Is the computation graph an acyclic graph or cyclic graph?
- What is the memory cost of gradient descent?
  - $O(\dim z + \dim w)$
- What is the computation cost of gradient descent?
  - Matrix multiplication  $w_i^{1\top} z^1$  (vector-vector):  $O(\dim w_i^1)$
  - Activation function  $\sigma(w_i^{1\top} z^1)$ :  $O(1)$
- Why can GPU speed up gradient descent?
  - Matrix multiplication  $Y = W^\top Z$ , where  $W \in \mathbb{R}^{k \times n}, Z \in \mathbb{R}^{n \times m}$
  - Parallelization:  $O(knm/c)$



(a) A directed cyclic graph.



(b) A directed acyclic graph.



# Auto-differentiation Tools

PyTorch



Tensorflow



JAX



# Why Deep Learning Now

- **Big data**

- Larger datasets
- Easier collection and storage

- **Hardware**

- Graphics Processing units (GPUs)
- Massively parallelizable

- **Software**

- Improved techniques
- New models
- Toolboxes

